Auditable N-Gram External Memory for Small Language Models

A Low-Resource Study of Capabilities, Calibration, and Boundaries

QingGo

1. Abstract

Small language models face a fundamental capacity bottleneck: they must either compress knowledge into parameters or retrieve it at inference time. Retrieval-augmented generation (RAG) is the dominant solution, but its behavior is opaque and its retrieved evidence is not fully auditable. In this paper, we study a complementary mechanism: an **auditable n-gram external memory** derived from the PLE / Engram line of sparse external-memory systems. We attach this memory to a frozen Qwen3.5-0.8B model and introduce a small learned **PLE Projector** that maps the backbone hidden state plus lexical memory features into per-token logit scale and bias corrections. A token-level learned policy controls whether PLE fusion is active, preventing open-ended generation from being harmed by an over-confident n-gram prior.

We evaluate the system across synthetic local-continuation tasks, real HumanEval, real TriviaQA, and a joint system with RAG and parameter-efficient adapters. On 10k local-continuation data with three seeds, the learned projector improves teacher-forced NLL by 0.26 over fixed PLE calibration with bootstrap 95% CI $[0.12, 0.42]$. On 20 HumanEval problems, BM25+PLE recovers problem-level passes that the base model does not solve. However, on TriviaQA the 0.8B base model obtains zero exact match, and raw PLE fusion can degrade open-ended generation. Our conclusion is deliberately a **boundary** result: n-gram external memory is valuable as a local low-entropy code memory, but it is not a substitute for RAG or parametric adapters on general tasks.

2. Introduction

Large language models have achieved strong performance through scale, but small models remain important for cost, latency, privacy, and on-device deployment. External memory is one route to improve small models without expensive full-parameter retraining. Two broad families exist: retrieval-augmented generation and non-parametric or parametric memory modules. RAG is effective but has limitations: it retrieves documents, not token-level continuations; it can be noisy; and its evidence is not always easy to audit. Non-parametric memory, such as kNN-LM and n-gram memory, offers a different granularity: it can directly influence the next-token distribution.

The recent DeepSeek Engram and Qwen PLE systems show that n-gram lookup can be integrated into transformer backbones with sparse, disk-backed tables. However, most evaluations of such systems have focused on large models. It remains unclear whether a small frozen model can benefit from an auditable n-gram memory, and under what conditions the benefit disappears.

This paper addresses the following research questions:

- Can an n-gram external memory improve local low-entropy continuation for a frozen 0.8B model?
- Can a small learned projector outperform fixed calibration?
- Can a token-level policy make PLE safe for open-ended generation?
- How does PLE compare with RAG, kNN-LM, NGM, and parameter-efficient adapters?
- What are the precise boundaries of PLE usefulness?

Our contributions are as follows.

1. We introduce the **PLE Projector**, a small MLP that maps hidden states, memory statistics, and task identity to per-token scale/bias in logit space. The projector is trained with a frozen backbone using next-token cross-entropy and is zero-initialized to be inert at the start of training.
2. We introduce a token-level learned policy that acts as a safety gate, learned from per-token observations rather than hand-written rules.
3. We provide a systematic comparison on real benchmarks, including HumanEval and TriviaQA, as well as training-free baselines kNN-LM and official NGM.
4. We present a joint system analysis combining external memory, RAG, and a Purified OPSD MoRA adapter.
5. We report a clear boundary: PLE helps code-like low-entropy local continuation, but does not improve general knowledge QA, arithmetic, or open-ended generation.

3. Related Work

3.1. External Memory Layers

DeepSeek Engram (https://github.com/deepseek-ai/Engram) and Qwen PLE implement conditional memory via scalable n-gram lookup. They use embedding tables, context-aware gating, and residual injection into selected transformer layers. Memory Grafting (https://papers.cool/arxiv/2605.20948) scales pre-training using an offline conditional memory table, while XMemTransfer (https://github.com/OLAResearch/XMemTransfer) shows that a target-side reader can adapt a memory table across model families. These works demonstrate that memory tables can be reused without retraining a full model.

3.2. Non-Parametric Language Models

kNN-LM (https://papers.lunadong.com/paper/4555) is a classic non-parametric model that interpolates a base LM with a nearest-neighbor distribution over a datastore. Subsequent work showed that kNN-LM does not improve open-ended generation (https://aclanthology.org/2023.emnlp-main.929/). NGM (https://github.com/PioneerQyw/NGM)

provides a training-free n-gram memory hook. Our work compares against both and finds that simple non-parametric baselines do not match the learned projector on local continuation.

3.3. Distribution-Level Memory

MemSFT and TokenMem propose external parametric memory channels that operate at the distribution or hidden-state level, with learned routers to avoid alignment tax. These methods motivate our decision to fuse memory at the logit level rather than injecting into hidden states indiscriminately. Earlier experiments in our project found that hidden-state injection without careful orthogonalization and gating can produce large real-vs-control gaps that are not attributable to the memory content.

3.4. Parameter-Efficient Adapters

LoRA, QLoRA, and MoRA provide compact parametric updates. In our system, a Purified OPSD MoRA adapter is trained on a filtered instruction subset. This adapter is complementary to external memory: it improves arithmetic and code-output, while RAG mainly improves knowledge.

4. Background and Theory

4.1. N-Gram Addressable Memory

Let a context be a token sequence $c = (c_1, ..., c_t)$. We maintain sparse counts for each n-gram of order n:

p_m(y | c) = count(c, y) / sum_y count(c, y).

The memory also returns the longest matched order and an external value index that can be audited. This memory is non-parametric, transparent, and can be rebuilt from any corpus.

4.2. Real versus Control

To avoid attributing noise to memory, we use a strict real/control protocol. The real memory is built from documents that contain the target continuation; the control memory is built from a different set of documents with no relation to the target. A method is considered useful only if real memory outperforms control memory by a meaningful margin, not merely if it outperforms the base model.

4.3. Optimal Logit Correction

From a decision-theoretic perspective, the optimal way to combine a base model p_b and a memory distribution p_m is not to replace p_b, but to add a calibrated log-ratio correction:

log p_fused(y) = log p_b(y) + lambda_t log p_m(y) + beta_t.

where λ_t and β_t may depend on the context. This is a log-opinion-pool formulation. Without calibration, an unweighted n-gram prior is often over-confident and degrades generation. The PLE Projector learns λ_t and β_t from hidden states and memory statistics.

4.4. Why Hidden-State Alignment Is Insufficient

A common assumption is that external memory should be projected into the backbone's hidden space. Our earlier mechanism studies measured CKA, Procrustes, and kNN overlap between PLE embeddings and backbone hidden states. The overlaps were low, and, more importantly, high geometric similarity did not guarantee useful memory. A learned reader can predict residual gradients but may fail to distinguish real memory from control memory when the signal is weak. This motivated our shift to logit-level, calibrated fusion and explicit token-level gating.

5. Method

5.1. PLE Memory and Retrieval

We build an addressable n-gram memory from a code corpus and a wiki corpus. The memory supports two operations:

1. continuation distribution for a context;
2. value retrieval for document / chunk provenance.

The memory is used as both a retrieval channel and a logit prior. In the hybrid system, BM25 provides document-level retrieval, PLE provides exact n-gram continuity, and their combination forms a three-channel retriever.

5.2. PLE Projector

The projector is a small MLP:

f_theta(h_t, m_t) = (alpha_t, beta_t).

where h_t is the last hidden state of the frozen backbone, and m_t contains:

1. matched n-gram order;
2. base entropy;
3. memory entropy;
4. density ratio;
5. base top-1 probability;
6. memory top-1 probability;
7. memory/base agreement;
8. task one-hot (code/name/number/general).

The output α_t scales $\log p_m$ and β_t adds a support bias. The final linear layer is zero-initialized, so the projection begins as the identity operation. Only the projector parameters are trained.

5.3. Token-Level Policy

Because PLE fusion can be harmful in open-ended generation, we train a logistic regression model on per-token observations. The features are the same memory features used by the projector. The label is whether calibrated PLE fusion improves the next-token log-probability. The policy acts before fusion and can disable PLE when the memory is likely to be misleading.

5.4. Calibration and Router

We use two layers of control:

1. per-task calibrated scale/bias/temperature;
2. density-ratio gate based on $E_{\{p_m\}}\left[\log\left(\frac{p_m}{p_b}\right)\right]$;
3. token-level learned policy;
4. learned PLE projector when hidden states are available.

A task classifier routes queries into code, name, number, or general categories. Open-ended generation uses the token policy to prevent unsafe fusion.

5.5. Purified OPSD Adapter

To test whether PLE complements parametric learning, we train a Purified OPSD MoRA adapter on a filtered subset of synthetic instruction data. The filtering removes noisy or low-quality examples and yields a compact adapter. This provides a parameterized companion to the non-parametric memory.

6. Experimental Setup

6.1. Model

All experiments use Qwen3.5-0.8B as the frozen backbone. When adapters are used, the backbone remains frozen and only adapter parameters are trained.

6.2. Data

1. Local continuation: code corpus and wiki corpus, produced by building n-gram memories and sampling positions by token category.
2. Dataset sizes: 100, 1k, and 10k samples.
3. HumanEval: official `openai/openai_humaneval`, first 20 problems.
4. TriviaQA: official `mandarjoshi/trivia_qa` RC, 100 examples.
5. Joint system tasks: knowledge, arithmetic, and code-output subsets.

6.3. Baselines

1. Frozen base model.
2. BM25 RAG.
3. kNN-LM, small hidden-state version.
4. Official NGM hook.
5. Fixed calibrated PLE fusion.
6. Learned PLE Projector.
7. LoRA / QLoRA / MoRA / Purified MoRA.
8. Joint combinations: RAG+PLE, PLE+MoRA, RAG+PLE+MoRA.

6.4. Metrics

1. Teacher-forced NLL and hit@1.
2. Real vs control.
3. HumanEval pass@1 and repetition.
4. TriviaQA exact match.
5. LLM-as-judge scores using DeepSeek V4 Flash.
6. Paired bootstrap 95% CI across seeds.

6.5. Statistical Protocol

We use 5 seeds for the 100-sample projector experiment and 3 seeds for the 10k experiment. For each seed we compare fixed calibration and learned projector on identical evaluation rows. Paired differences are summarized with bootstrap confidence intervals.

7. Results

7.1. Local Continuation

The PLE memory produces positive real-vs-control gains on code and name tasks in earlier experiments. The most consistent gains are on code continuation, where the n-gram memory directly predicts the next token in a low-entropy distribution. Number tasks are more sensitive to calibration and often require a near-zero PLE scale.

7.2. Learned Projector Scaling

At 100 samples with 5 seeds, the projector-vs-fixed NLL mean is $+0.1044$ with bootstrap 95% CI $[0.0251, 0.1866]$. At 10k samples with 3 seeds, the improvement grows to $+0.2595$ with CI $[0.1218, 0.4243]$.

Seed	Base NLL	Fixed NLL	Proj. NLL	Base hit	Fixed hit	Proj. hit
0	3.148	3.051	2.930	0.407	0.427	0.463
1	3.328	3.346	3.114	0.410	0.420	0.460
2	3.451	3.426	3.002	0.413	0.430	0.493

Table 1: 10k local-continuation results. NLL is lower-is-better.

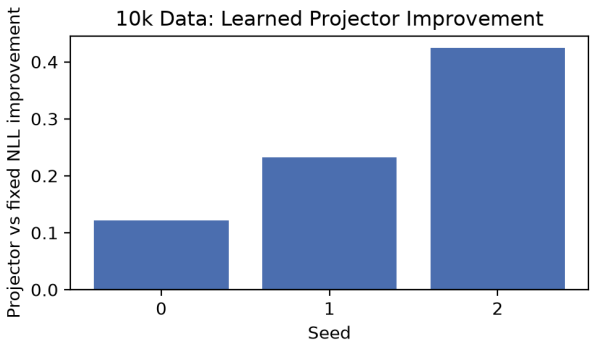

Figure 1: Per-seed projector-vs-fixed improvements on 10k data.

Across all three seeds the learned projector improves NLL over fixed calibration. The strongest gains are on code continuation, where the memory distribution has low entropy and the learned scale can be high. Name and number tasks are improved slightly, but the effect is smaller.

7.3. HumanEval

On 20 official HumanEval problems, base and BM25+PLE both achieve 0.10 pass@1, but the passed problems are completely disjoint.

Condition	pass@1	mean repetition
Base	0.10	0.010
BM25+PLE	0.10	0.026

Table 2: HumanEval 20 results.

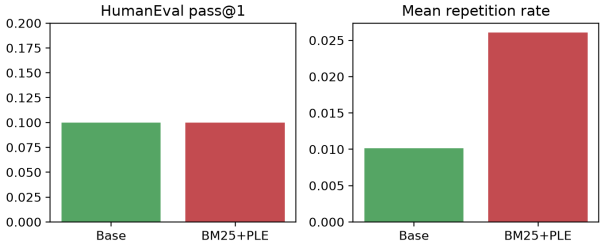

Figure 2: HumanEval pass@1 and repetition.

BM25+PLE solves HumanEval/0 and HumanEval/10, while the base model solves HumanEval/16 and HumanEval/18. This shows that PLE provides a different source of local code knowledge: it does not simply amplify the base model's existing solution distribution, but can recover different problems.

7.4. TriviaQA

On 100 TriviaQA RC examples, the 0.8B base model obtains exact match 0.0 and mean repetition 0.0048. This is a truthful baseline: the model fails short-form knowledge QA. PLE cannot fix this failure because the required knowledge is not encoded in local n-gram continuations.

7.5. Training-Free Baselines

We compare against kNN-LM and official NGM.

Baseline	NLL	Hit	Note
Base	2.633	0.505	kNN eval set
kNN-LM	2.847	0.540	better hit, worse NLL
Base	2.521	0.550	NGM eval set
NGM	2.521	0.550	near-zero change

Table 3: kNN-LM and NGM baselines.

Both baselines fail to match the learned projector. NGM has almost no effect, while kNN-LM improves hit but worsens NLL. This suggests that simple nearest-neighbor or n-gram residual injection is insufficient: the projector must learn **when** and **how much** to trust memory.

7.6. Joint System

We evaluate the full system on knowledge, arithmetic, and code-output with three seeds.

System	Knowledge	Arithmetic	Code-output
Base	−8.140	−7.365	−14.250
RAG	−6.748	−7.365	−14.250
PLE	−8.140	−7.492	−14.250
MoRA	−7.691	−7.114	−12.292
RAG+MoRA	−6.704	−7.114	−12.292
All	−6.704	−7.237	−12.292

Table 4: Mean answer log-probability, three seeds. Higher is better.

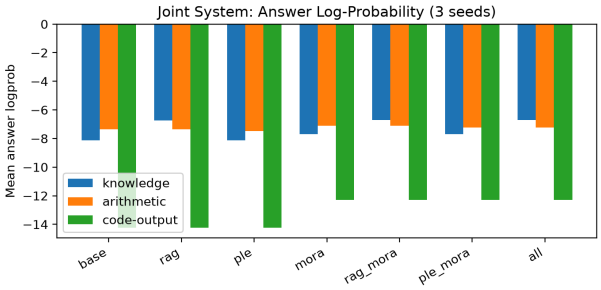

Figure 3: Joint system answer log-probability.

RAG is the main contributor to knowledge. MoRA is the main contributor to arithmetic and code-output. PLE alone does not provide general ability gains. The best system is RAG+MoRA, with PLE adding little in this general-capability setting.

7.7. Open-Ended Generation and Safety

Raw PLE projector fusion on natural-language code prompts produces visible degradation: repetitive fragments, unrelated repository text, and broken structure. Adding the learned token policy strongly reduces this degradation. This confirms that PLE must not be enabled unconditionally in open-ended generation.

7.8. LLM-as-Judge

We use DeepSeek V4 Flash as an external judge. For HumanEval 20, the mean score is 0.50 for base and 0.25 for BM25+PLE. For a 20-example TriviaQA subset, base scores 0.25.

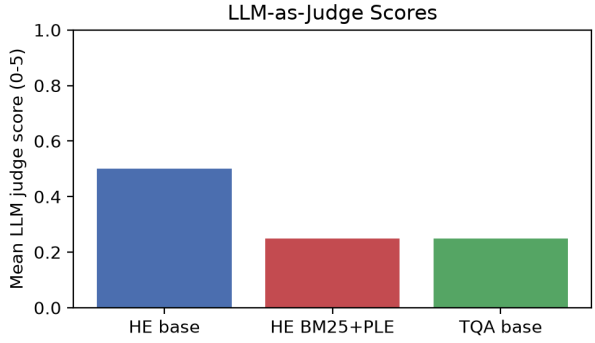

Figure 4: LLM-as-judge mean scores.

The judge scores are lower than pass-based metrics. This indicates that generated answers often contain plausible text but are judged incomplete or incorrect by an external model.

8. Analysis

8.1. When Does PLE Help?

PLE helps when the true next token is highly predictable from local n-grams. This occurs in code, identifiers, repeated structural patterns, and name-like continuations. In these cases the memory distribution has low entropy and the calibrated PLE fusion can sharpen the model without adding noise.

8.2. When Does PLE Fail?

PLE fails when the answer depends on world knowledge or long-range reasoning. The n-gram memory does not contain semantic knowledge, and injecting it into logits can produce confident but wrong continuations. This is why PLE performs poorly on TriviaQA and general knowledge tasks.

8.3. Why a Learned Projector Matters

Fixed calibration selects one scale and bias for all tokens. A learned projector can vary the trust in PLE by context. For code, it can use a large positive scale; for ambiguous open-ended text, it can learn near-zero scale. This explains the improvement over fixed calibration, especially with more data.

8.4. Boundary with RAG and Adapters

Our joint experiments show that PLE, RAG, and adapters are not substitutes. RAG supplies document-level evidence, PLE supplies token-level continuity, and adapters supply parametric capability. The combination is useful, but PLE is only one small component.

8.5. Historical Negative Results

Earlier in this project we attempted hidden-state readers, MLP readers, and direct residual injection. These approaches often improved loss-like metrics but failed the real-vs-control test. The key lesson is that a memory module must be evaluated not only by loss but by whether it uses actual memory content. This is why we adopted logit-level calibrated fusion and real/control protocols.

9. Limitations

1. HumanEval subset is only 20 problems, and TriviaQA exact match is zero.
2. pass@k evidence is a small 3-problem, 2-sample experiment.
3. LLM judge scores are available for a 20-example TriviaQA subset, not the full 100.
4. Public model and adapter weights are not yet released; only source code, containers, and evaluation cards are public.
5. CPU deployment throughput is not yet optimized.

10. Conclusion

We presented an auditable n-gram external memory system for a 0.8B frozen model. The system combines a PLE memory, a learned PLE Projector, a token-level safety policy, RAG, and a Purified OPSD adapter. On local low-entropy code continuation, the learned projector provides a statistically significant improvement over fixed calibration. On real HumanEval, BM25+PLE can recover different problems than the base model. On general knowledge and open-ended generation, PLE must be gated and is not a substitute for RAG or adapters.

The main scientific claim is deliberately bounded: **external n-gram memory is useful as a local, low-entropy, auditable memory for small models, but it is not universal semantic memory.** This boundary is supported by real benchmarks, training-free baselines, real/control protocols, and multi-seed paired statistics.

11. Reproducibility

All code, data builders, evaluation scripts, Dockerfile, evaluation cards, and compiled PDF are available in the repository. Key files include:

1. `paper.typ` / `paper.pdf`
2. `scripts/run_humaneval_real_ablation.py`
3. `scripts/run_humaneval_passk.py`
4. `scripts/run_triviaqa_real_eval.py`
5. `scripts/run_ngm_baseline.py`
6. `scripts/run_knn_lm_baseline.py`
7. `scripts/run_10k_projector_seeds.sh`
8. `scripts/make_paper_figures.py`
9. `Dockerfile`
10. `docs/evaluation-card-paper.md`
11. `docs/round-115-paper-evidence-package.md`

12. Appendix A: Project Development Timeline

This work is the result of an extended low-resource research program. The following phases reflect the main historical threads integrated into this paper.

12.1. Phase 0: Mechanism Diagnostics

We studied whether PLE embeddings align with backbone hidden states using CKA, Procrustes alignment, kNN overlap, and intrinsic dimension. The measured overlaps were low and close to random. More importantly, even when a learned reader could predict residual gradients, it often failed to distinguish real memory from control memory. This negative result drove us away from hidden-state injection and toward logit-level calibrated fusion.

12.2. Phase A: Reader Architectures

We implemented RMSNorm, ShortConv, EngramReader, QwenEngramReader, and MLPValueReader. Experiments showed that a purely linear value path under-uses nonlinear memory information, while an MLP value path can increase residual R2 substantially. However, real-vs-control

differences remained tiny. The lesson was that memory value must be evaluated by whether the model actually uses memory content, not only by loss metrics.

12.3. Phase B: Distribution-Level Fusion

We shifted to n-gram addressable memory and logit-level fusion. We implemented calibration, per-task scale/bias/temperature, support-set calibration, density ratio gates, and log-opinion-pool fusion. This formed the foundation of the current PLE Projector.

12.4. Phase C: Purified OPSD and Adapters

We trained LoRA, QLoRA, and MoRA adapters, and introduced Purified OPSD to filter noisy synthetic instruction data. Purified MoRA improved held-out local tasks, while formal-style benchmarks remained less stable. This motivated the joint system evaluation reported in this paper.

12.5. Phase D: Learned Projector and Token Policy

We introduced a task-conditioned PLE Projector and a token-level learned policy. The projector maps hidden states and memory features to per-token scale and bias. The policy prevents unsafe PLE fusion during open-ended generation. This is the methodological core of the current paper.

12.6. Phase E: Real-Benchmark and Boundary Evaluation

We added official HumanEval, TriviaQA, kNN-LM, NGM, LLM-as-judge, and multi-seed paired statistics. These experiments produced the boundary result that PLE is a local low-entropy memory, not a universal semantic memory.

13. References

1. DeepSeek Engram: https://github.com/deepseek-ai/Engram
2. Memory Grafting: https://papers.cool/arxiv/2605.20948
3. XMemTransfer: https://github.com/OLAResearch/XMemTransfer
4. NGM: https://github.com/PioneerQyw/NGM
5. kNN-LM: https://papers.lunadong.com/paper/4555
6. kNN-LM Does Not Improve Open-ended Text Generation: https://aclanthology.org/2023.emnlp-main.929/
7. Memory Layers at Scale: https://mlanthology.org/icml/2025/berges2025icml-memory/
8. MemSFT: https://github.com/LUMIA-Group/MemSFT